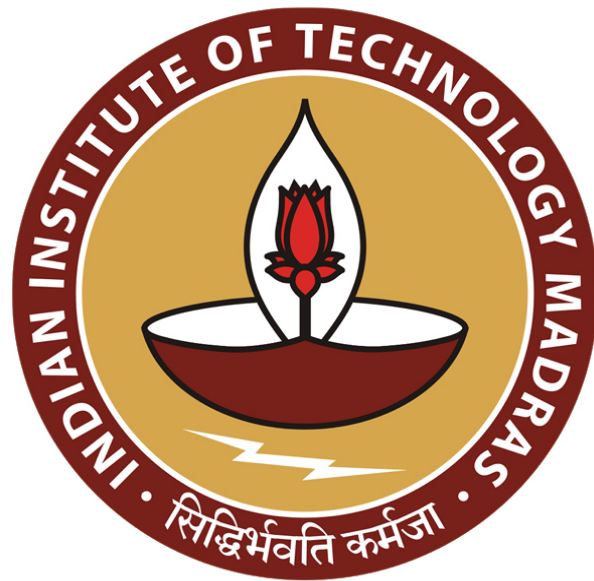




IIT Madras
ONLINE DEGREE

Machine Learning Techniques
Professor Arun Rajkumar
Department of Computer Science and Engineering
Indian Institute of Technology, Madras
Issues with PCA

(Refer Slide Time: 00:13)

REAL WORLD EXAMPLE

EXTENDED YALE FACE DATASET

EACH Data point $x_i \in \mathbb{R}^d$
 $d = \sim 32000$; $n = \sim 20000$

PRINCIPLE COMPONENTS

PC1 PC2 ... PCk

[Row of face images] [Row of face images]

Hello, everyone. Welcome back. So, today, what we will do is continue our discussion on PCA. And last time, we said that we will look at one example for PCA, (real time application), real world application. And we will also look at some issues with PCA. And that will lead us to something more interesting, which not only fixes the issue, but also gives us a general idea which can be used for many supervised learning algorithms that we will see later on.

But before going there, let us start with very interesting, real world application of PCA which is called as the Eigen faces application. So, what is this application? So, to look at this application, let us look at this example. So, this is a real world example, which is called as the YALE FACE DATASET, the extended YALE FACE DATASET.

And the reference to this dataset is in the right here in the Washington University's website. So, it is just a bunch of faces, images of faces, which are of the same size, so all facing the camera images, all grayscale images, and around 20,000 of them. So, that is what this dataset contains. Now, how do you convert an image into features?

Well, the way that this is done for the Yale dataset is a very simple idea, you take, you treat the image itself as a bunch of pixels. So, basically, so each image consists of some $m \times n$, you can think of it as an $m \times n$ matrix of pixels. And each pixel value is a grayscale value, which is something some number between 0 and 255.

And now you can, vectorize, this whole matrix $m \times n$ matrix into a vector of size $m \times n \times 1$. So, it is a, so when you do that, so there are around 32,000 pixels, I am not putting the exact number here it is around 32,000, maybe slightly more, that does not really matter for our discussion here. But there are around 32,000 pixels in each image.

So, each image becomes a point in a 32 dimensional space. Now the question is, well, if you want to get a compressed representation of this data set, so can we use PCA to do this, basically, you treat this as a data set containing 20,000 points, 20,000 faces, and each point is a vector in 32,000 dimensional spaces.

Now, what you could do is just run your standard PCA, whatever we have seen so far, that is compute the covariance matrix, look at its Eigen directions, and each Eigen direction is now going to be in the feature dimension. So, the covariance is a $d \times d$ matrix. So, which means there will be a 32,000 cross 32,000 matrix, if you look at the Eigen vectors for it.

So, they will also be in 32,000 dimensional spaces. So, the best line that fits will be a line represented by a vector in 32,000 dimensions. Now, what you could do is the following, you can

now again convert this 32,000 dimensional vector back into an image and see how that looks like.

So, the way we went from images to 32,000 dimensional points was map this $m \times n$ to an $mn \times 1$ dimensional vector. Now, we can do the same thing in the reverse as well. So, because there is a specific ordering for these coordinates, where you converted your $m \times n$ to mn dimensional vector, you can do the you can go back as well.

And if you did that, we can ask how do, these principal components look like if I visualize them as images. And when you do that, for this dataset, you see, the principal components look like these. So, basically, the first principal component looks like this image here. The second principal component looks like PC2 second image and so on.

Now, let us pause for a second and see what do these principal components intuitively mean? So, why are they looking like the way they are looking? So, basically, PC1 kind of captures that direction, which has the most variance in the data set. That is what we saw PCA does earlier. Now, if you convert this into an image.

So, it is as if this is like the best prototype image that captures most of the variance in your data set because this is a dataset full of human faces. So, now this image captures the faceness of this data set in the best possible way, you could say that. Now as you look at further more principal components, what you observe is that you get more and more details that get added to these directions.

So, for example, PC K, which is some Kth principal component, now has a much more, in some sense a nuanced representation, it captures much more details. So, there is, the noses are more clearer the mouth is more clear and so on. So, now, why does this happen, because, what is common for most of these data points is this template, is the most common template.

In some sense, this is where most of the variance is, but then as you look at other directions, they are trying to find out other characteristics of this data set, which are also important, which is where more and more details get added, as you increase more and more principal components. Now, the question is, what do we do with this dataset?

One question is, so let us say if you want to build a, like a face recognition system, for the employees in a company or something like that, where you have 1000 employees, so, in this case, 20,000 employees

Now do you really want to store 32,000 numbers per employee to capture that person's photograph, so 32,000 pixel details? Do we really need that? Or is there a lower dimensional representation that perhaps, represents what we want really well? So, now PCA would tell us if that is the case.

So, now, what we are trying to do now, what we will try to do now is we will increase the number of principal components that we will retain or in other words, the number of rounds, we will run the algorithm for. Or in other words the number of Eigen directions, that we are going to retain all of these are exactly the same things in our context.

And now, we will try to reconstruct the image, the proxy for, this image using only these retained principal component. In other words, what does this mean? This means that if I just retained one principal component, then for each of these images, I am going to take that image dot product with the first principal component that will give me a number.

And now to reconstruct the proxy, I will multiply this image with just that number. So, all the images will be some scaling of this number. Now, if I am reconstructing with two images, it means that I will take the dot product with the top two Eigen directions, principal components, and then I linearly combine these dot products, using with these images to get a new image.

So, now the question is, as I increase the number of Eigen directions, how do these images start looking like and that the Yale folks did that as well. And what you see is something like this? So, for example, if you let us focus on the left hand side image, first now here is a test image. So, let us say this is an image that is not in your dataset, maybe again, in a face recognition context, this could be some person who walks into your company.

And then your inbuilt CCTV camera or something like that clicks a picture of this person, and then you get an image of this sort. So, now, what you are trying to do is, you are trying to reconstruct this image using only the top few principal components. This is the image reconstructed using just 25 principal components, the next one using 50, 100, and so on.

Now, focus on the original image, and the reconstruction using 1600 principal Components, that is pretty much close. So, you can already tell this is exactly the same person. In fact, you do not really need 1600, well, is 25 enough? Perhaps not.

So, if you only use 25 components, it is not very clear if this is exactly the same person, it is clear that it is a person. But it is not clear if it is exactly the same person. But then if I use 200, components, let us say, or even 300, 400, or somewhere between 200 and 400, pretty much all the important details of this person specific to this person is also captured in some sense.

So, you only need something like 300 directions to capture this person's, essential features that we need to say if this is the person or not, you do not necessarily need all the way to 1600. Well, if you go to 1600, you are adding more and more, finer details. So, maybe there are more details about how the faces how the eyes and noses are, and so on, which is perhaps not necessary to really say if this is the same person or not, only 300 numbers are sufficient.

So, which means to say that the original image had 32,000 pixels, but now what are we saying? We are saying that well, just 300 numbers, you can perhaps recognize each of these persons pretty well. You do not need 32,000 numbers. So, that is like 100 times again, that you can get by doing this PCA. Well, here is another way to look at this.

So, here is a say a different image, which is not characteristic of the data set from which (you are running) your running PCA and getting the principal components. So, this is a dog image. So, now this also has 32,000 pixels. Now, what are we doing, we are only taking the top 25 and trying to reconstruct this dog, the proxy for this dog in that 22,000 dimensional space that best represents this dog.

But because the 25 dimensional space that best represents a dog that is what we are trying to find and because these 25 directions are the top 25 directions with respect to human faces, if you are trying to project our dog onto this 25 dimensional human space, you are not really going to see that dog reconstructed that well.

So, you are only going to see something like, the best human representation of this dog. So, that is what you are essentially getting. You are not really getting the dog reconstruction that well, now as you keep going, so for r equals 100, well, still not that good for r equals 200 still pretty

bad, 400 is pretty bad. 1600 still. So, when you have 1600 dimensions, (then the dog), the reconstructed dog pretty much looks like the dog.

Now, the point I am trying to make here, and this is important in understanding what is happening here is that, because the data set on which we got these principal components from content, only human faces, it is trying to capture the most important directions with respect to human faces, which means if you start with some other the dogs face, or even a tree, you are not going to get good reconstruction with small number of principal components.

So, if your goal was to distinguish a man, or a face, man or a woman, from a dog, so a human from a dog, then you do not necessarily perhaps need even 200 components. I mean, even very small number of components is good enough. Even with 25 components, you can say if it is a human face, or a dog face.

So, because the dogs faces with on the 25 dimensional subspace looks nothing like human or nothing like dog, so you still might get some distinguishing capability. On the other hand, if you really want to say it, is this particular human, well, you still do not need 1600. So, you only need something like 200, 200 to 300.

But then with 200 to 300, the dog does not look anything like the dog, with 1600 you are somehow able to reconstruct both the human faces and the dog's face correctly. So, which means there is enough information in the top 1600 directions that can represent perhaps any face. So, anything that has eyes, nose, and mouth, perhaps is captured in that, 1600 dimensions.

So, you do not, now you would not really be able to, I mean the dog looks like the dog, the human looks like the human. So, with 1600, you kind of get most of the information. Whereas if you took a tree, perhaps you would be much more dimensions to reconstruct the tree 1600 may not be enough, or if you just took a random image so, an image which where each of the pixel values were absolutely random.

Now, how many dimensions do you think , you would need to reconstruct this image? Of course the image does not have any structure. So, it would not even be possible to say if the reconstructed image is similar to the original image or not. But then if you want to exactly

reconstruct this, you would need all 32,000 dimensions. But remember, if you have 32,000 dimensions, you can reconstruct any image so, because you have complete information.

So, for any set of 32,000 , dimensional vector, what we are asking is with smaller numbers, when can you reconstruct well? Well, if it is a human face, you can reconstruct well, with just 200 to 300 dimensions, if it isa dog face you perhaps need 1500 to 1600. If it is a tree, maybe you need 2000 3000. I do not know, I have not tried it on this dataset, but that would be my guess. But if it is a random image, you would need more.

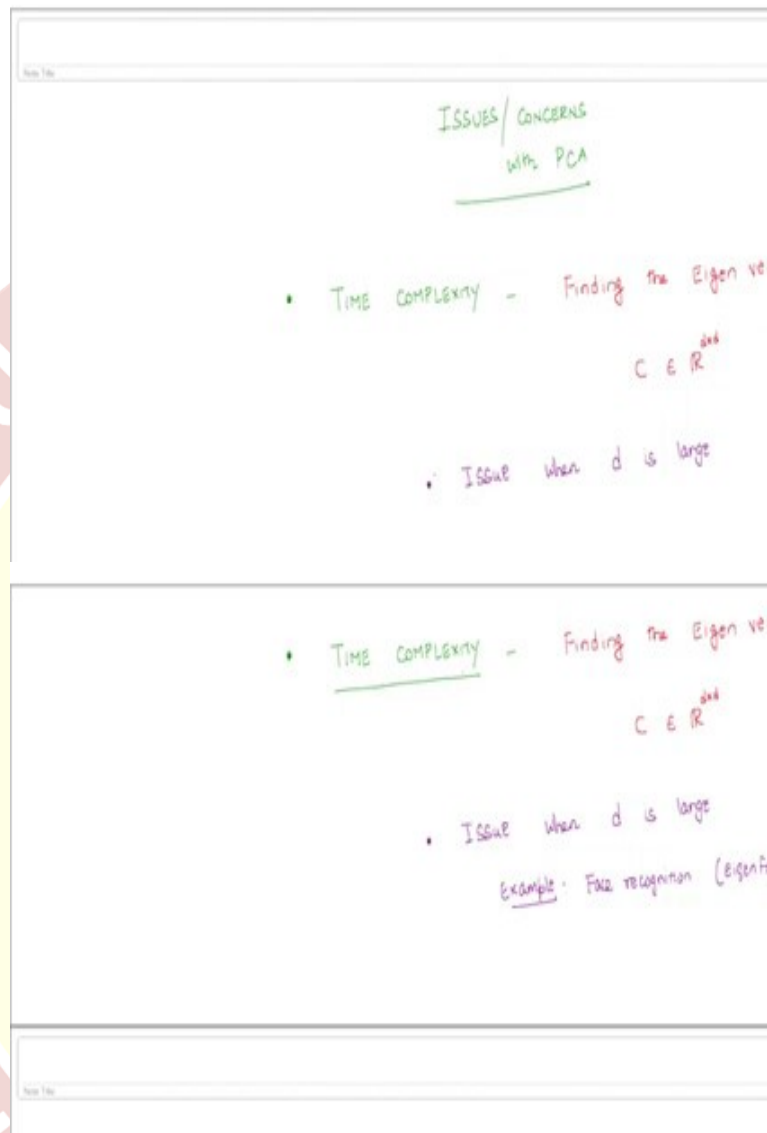
So, 32,000 in the best the worst case so, you would need to store them. So, this is a very good example of PCA which is called Eigen faces, which can recognize faces, specifically human faces. Of course, we will use this as like a pre processing technique to store only the top projections on the top few Eigen directions, and then use it later on to train a supervised learning algorithm. That is the basic idea.

So, given a new test image, you will again project it onto the top few Eigen directions with respect to the human faces and then run your supervised learning algorithm to see if the, if you got the correct answer or not. And, because this works, because there is some structure to these natural images so, there is some underlying subspace , that is the technical word to use.

So, there is some underlying linear subspace where most of the information is captured of the human faces. And so, this algorithm works. So, that is a nice, example of an application of PCA, there are many more and just wanted to show you one example. Now, let us continue our discussion to understand what are some, shortcomings of the PCA algorithm itself and how we can fix those shortcomings.

And as I said, the more we try to understand these shortcomings, and then try to fix them, that will lead us to something very, very interesting, which is useful mode in, I mean, in general for machine learning applications.

(Refer Slide Time: 16:09)



So, the next thing that we are going to look at is issues or concerns with PCA. So, we will, point out two main issues or concerns with PCA and then try to fix both of them in the course of time, and the first issue, it is not like a problem, it is but then it is like a concern definitely is the time complexity. So, how much time does it take to run PCA?

So, we have an algorithm. As computer scientists typically, you should care about how long would, it take to run this algorithm. So, what is the most time consuming step in PCA? Well PCA is just constructing a covariance matrix and then computing the Eigen directions of it. So, finding the Eigen vectors is the time consuming step of finding the Eigen vectors and Eigen

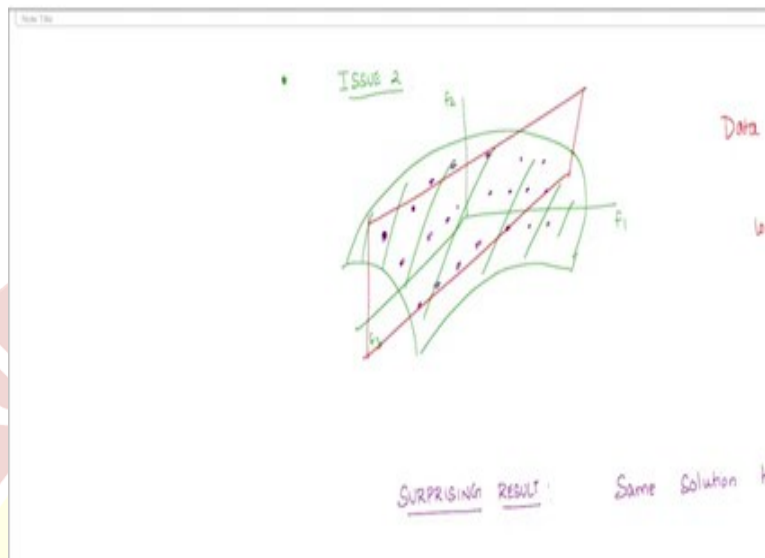
values of covariance matrix. Now, there are a lot of special purpose numerical linear algebra solvers, which can perhaps do this in the most efficient way possible. But what I am going to suggest now is a general time taken for finding the Eigen values and eigenvectors of any matrix, in particular the covariance matrix.

So, we care about the covariance matrix, which is a $d \times d$ matrix. And typically, let us say typically, because, of course, there might be special purpose softwares which do this faster, but usually it takes order of d^3 time, so, this is the time complexity. If you are not familiar with order notation, that is fine.

So, you can think of this as d grows, d is the dimension of the data set, the time taken grows cubic in d . So, it would be order of d^3 . So, this is a problem this is an issue when d is large. So, when d is large, so, question is if d the number of features is large, for instance, in the Eigen faces applications that we saw the number of features was around 32,000 the number of data points was smaller than the number of features but only perhaps 20,000 or something like that.

So, if the number of features is large, d is orders of 10s of 1000s or even millions, then d^3 is going to be a million cube, which is huge, perhaps huge for computing these Eigen vectors and Eigen directions. So, an example of where this can happen is face recognition. Eigen faces, so, that we saw where the number of features was 30,000 this is problem number 1. So, we would not really say at this point how to fix it. We will talk about this in a minute, but then I will point out one more issue and then revisit this. What is problem number 2?

(Refer Slide Time: 19:43)



Problem number 2 is an unrelated problem to problem number 1, at least at this point, we will feel like that. This is issue 2. Which is - so what is PCA trying to find PCA is trying to find some linear combination of these features, which are very important. So, if there is weight, if there is height, maybe weight plus height is a very important feature, or two times weight plus three times height is very important, and things like that.

But perhaps let us say you had weight, height and something like a body mass index or something like that. Now, the body mass index might not linearly depend on weight and height, maybe it is, it is something like weight over height squared, I might get the formula wrong, but then there is a quadratic dependency with respect to how the body mass index is related to the weight and height. So, it is not linear.

So, which means that, if we for instance, if you plot in 3-D, Feature 1, Feature 2, Feature 3, and let us say we observed that the data points fell something like this. Now, what does, what am I trying to allude to here is that it could be the case that, you have some manifold, or some curved space, which is a subset of your original space, which is R^3 .

So, this is a curved space. And, all your data points lie along the curved space, let us say, now, the fundamental structure in your data is not a linear structure. Why is it curved? It is curved

because these features themselves have a nonlinear relationship. Every F_3 is $F_1 \times F_2 + F_2^2$ or something like that.

So, maybe in your application, that is what is the fundamental equation that relates these features or the most important equation, now this is a structure, just that PCA would not find this, which because PCA is not looking for these kinds of relationships. PCA is only looking for relations which are linear. So, how can you linearly combine?

How can you weight each of these features, add them up and then create new features? Not all applications may have such relationships. In other words, so if you run a PCA, so you might get something, you will still get some output. So, you will get let us say, if this is like a 2 dimensional manifold inside a 3 dimensional space, let us forget the technical definition of manifold. It is a curved space for us.

But then essentially, think of it as a blanket, which is curved, and all the points are on this blanket. But now if you run PCA, you are still going to get some results. So, PCA might, I mean, there is nothing stopping us from running PCA on this. PCA might say that there is some plane, which is the best plane that represents this data.

Now, if you compute your error with respect to this plane for these data points, that error is still going to be high, it is not going to be insignificant, which means that we might be fooled into thinking that well, there is a third dimension, which is also important for this data set. And we might add one more dimension of course then you will be able to exactly reconstruct this dataset if you have 3 dimensions, that is, you cannot go further I mean, you can reconstruct any data set.

So, error will be 0, but then PCA will not know that there is a fundamentally mean essentially a 2 dimensional structure that is, hiding in this data set. So, the problem though is the data may not necessarily lay or lie in a low dimensional and the key word is not just low dimensional, low dimensional linear subspace.

So, when I say subspace, I just mean can think of it as 2 dimensional planes passing through origin, or line passing through origin and the higher dimensional analog of those. So, both of these issues that we have identified are relevant issues. One is time complexity, when the number of features is large you have a time complexity issue.

The second is the features themselves are non-linearly related. Then also you have an issue. So both of these cases, in one case, PCA will work but then will take a lot of time. In the other case, PCA will work but then may not give you the desired result that you really need. Here is a surprising fact. And this is perhaps the punch line of this video is that we will have the same solution to both these issues.

At this point, this might seem odd. So, these two issues do not seem related at all. One is the time complexity issue, the other is a question about the structure. And, the surprising thing is that if you solve one, you have actually solved the other as well, in the sense that you can use the solution to one to intelligently to solve the second problem as well.

We would not break the suspense as to how that is done at this point. But what I will do is we will first consider issue 1, try to solve that and then make some observations which will help us in solving issue 2 also. So, let us start with the solving issue 1 and how to go about issue 1.

